DISEÑO DETALLADO DEL EJERCICIO

Predicción de la Pobreza

mediante Imágenes de Google Street View y Redes Neuronales Convolucionales

Localidad de USME · Bogotá D.C. · Abril 2026

Curso MECA 4107 — Big Data y Machine Learning para Economía Aplicada

Profesor: Álvaro Riascos Villegas · Universidad de los Andes / Quantil

Visión General del Ejercicio

Este documento explica paso a paso un ejercicio de machine learning aplicado a economía del desarrollo. El objetivo es usar fotografías de calles de USME (Bogotá), tomadas por el vehículo de Google Street View, para predecir qué tan pobre es una zona. Esto se hace entrenando una red neuronal artificial que aprende a asociar lo que ve en una imagen con un índice de pobreza medido por el DANE.

Cada paso se explica en términos cotidianos y se complementa con las fórmulas matemáticas que corresponden a lo visto en los materiales del curso.

Figura: Flujo completo de los 6 pasos del ejercicio

La pregunta central

¿Se puede estimar con fotografías de calles bogotanas qué tan pobre es una zona? ¿Qué elementos visuales usa y son más importantes para eso? Este ejercicio responde ambas preguntas.

PASO 1	Cruce Geográfico — Asignarle un índice de pobreza a cada imagen

¿Qué problema resuelve este paso?

Las imágenes descargadas de Street View son solo fotografías. La computadora no sabe si esa calle es rica o pobre. Para que pueda aprender, necesitamos decirle, para cada imagen, cuál es el nivel de pobreza de la zona donde fue tomada. Ese número se convierte en la respuesta correcta que el modelo debe tratar de reproducir.

¿Qué es el NBI y por qué lo usamos?

El Índice de Necesidades Básicas Insatisfechas (NBI) mide el porcentaje de hogares en una zona que no cubre al menos una necesidad básica: vivienda adecuada, servicios sanitarios, ausencia de hacinamiento, asistencia escolar o capacidad económica mínima. El DANE lo publica a nivel de Unidad de Planeación Zonal (UPZ) para Bogotá. Un NBI de 30 significa que el 30% de los hogares de esa UPZ tiene al menos una carencia básica.

¿Cómo se hace el cruce?

Cada imagen fue descargada en una coordenada específica (latitud y longitud). Con geopandas, equivalente a un sistema de información geográfica simplificado, identificamos a qué UPZ pertenece cada coordenada. Luego le asignamos el NBI de esa UPZ. El resultado es una tabla donde cada fila tiene: la imagen y su NBI.

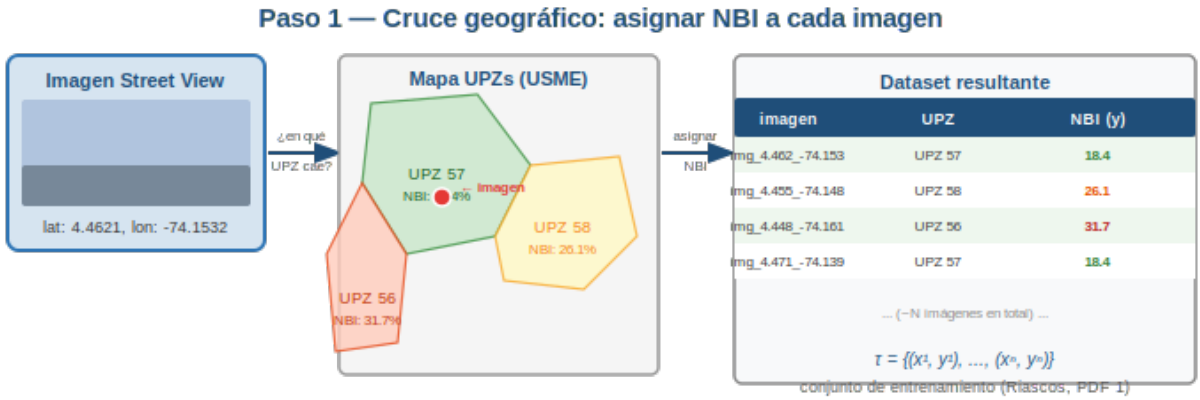


Figura: Cruce geográfico: cada imagen recibe el NBI de la UPZ donde fue tomada

Construcción del conjunto de entrenamiento

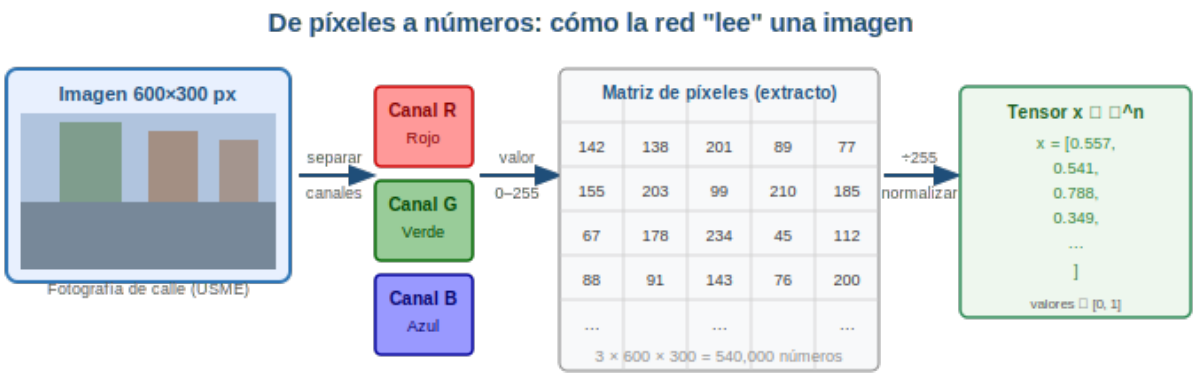
En el PDF de Introducción a Redes Neuronales (Riascos), el punto de partida de cualquier modelo de aprendizaje supervisado es lo que se llama el conjunto de entrenamiento:

$$\tau = \{ (x^1, y^1), (x^2, y^2), \ldots, (x^n, y^n) \}$$

Donde x^i es la imagen i-ésima convertida en números (ver Paso 2) y y^i es el NBI de la zona donde fue tomada. Este paso construye exactamente ese conjunto τ. Sin él, no hay aprendizaje posible.

¿Qué es una imagen para la computadora?

Una fotografía de 600×300 píxeles tiene 3 canales de color (rojo, verde, azul). Cada píxel en cada canal es un número entre 0 y 255. En total, cada imagen es un vector de 3×600×300 = 540,000 números. Ese es el vector x que entra a la red neuronal.



Cada imagen es un vector de 540,000 números. Eso es el "x" que entra a la red neuronal.

Figura: Cómo se convierte una fotografía en un vector numérico (tensor)

¿Por qué normalizar dividiendo por 255?

Los valores de píxeles van de 0 a 255. Las redes neuronales funcionan mejor cuando sus entradas están en un rango pequeño, típicamente entre 0 y 1. Dividir por 255 transforma cada valor al intervalo [0, 1] sin perder información. Esto acelera el entrenamiento y estabiliza los gradientes.

PASO 2 Preparación del Dataset y Data Augmentation

El problema: pocas imágenes para entrenar una red profunda

Las redes neuronales profundas tienen millones de parámetros (pesos) que ajustar. Para aprender bien necesitan muchos ejemplos: idealmente decenas o cientos de miles de imágenes etiquetadas. En nuestro caso tenemos las imágenes de USME, que cubren principalmente los corredores viales, lo cual puede ser un número limitado. Con pocos datos, el modelo corre el riesgo de memorizar en lugar de aprender.

La solución: Data Augmentation

El PDF de Tópicos de Implementación (Riascos) menciona explícitamente el data augmentation como técnica para aumentar la cantidad de datos: 'Para aumentar la cantidad de datos, variedad, etc. se puede usar: rotaciones, traslaciones, reflexiones. Acercamientos. Agregar ruido.'

La idea es simple: si tenemos una fotografía de una calle, podemos generar versiones modificadas artificialmente de esa misma imagen — reflejada horizontalmente, con el brillo ligeramente cambiado, rotada unos pocos grados, recortada y ampliada — y tratarlas como ejemplos adicionales de entrenamiento. El NBI asignado se mantiene igual porque la zona no cambia.

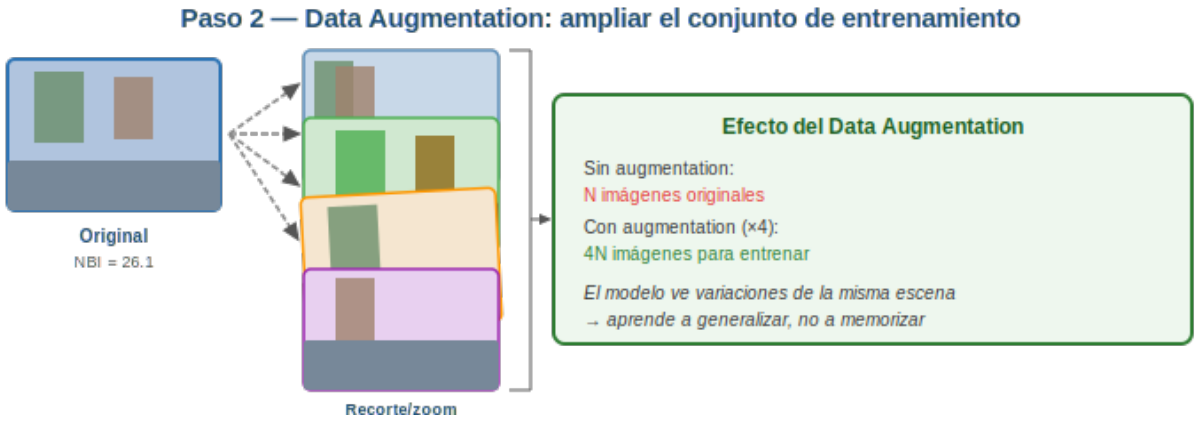


Figura: Data Augmentation: de una imagen original se generan múltiples versiones para entrenamiento

¿Por qué esto funciona?

Cuando el modelo ve una calle reflejada, o con menos brillo, aprende que su predicción de pobreza no debe depender de si la calle va hacia la izquierda o la derecha, ni de la hora del día en que fue tomada la foto. En otras palabras, aprende a ignorar variaciones irrelevantes y a enfocarse en las características que realmente importan: el estado de las fachadas, el tipo de pavimento, la presencia de infraestructura.

Normalización de imágenes — la media y desviación estándar de ImageNet

Adicionalmente, cada imagen se normaliza usando la media y desviación estándar calculadas sobre ImageNet, la base de datos con la que fue preentrenada ResNet18 (ver Paso 4). Esto garantiza que los valores de entrada estén en la misma escala que los datos con los que la red fue originalmente entrenada, lo cual es necesario para que el transfer learning (usar el modelo ya entrenado) funcione correctamente.

```
x_normalizado = (x - μ_ImageNet) / σ_ImageNet

μ_ImageNet = [0.485, 0.456, 0.406]  (por canal R, G, B)
σ_ImageNet = [0.229, 0.224, 0.225]  (por canal R, G, B)
```

PASO 3 División Train / Validation / Test con split espacial

¿Por qué no se pueden usar las mismas imágenes para entrenar y evaluar?

Imaginemos que estudiamos para un examen leyendo las respuestas exactas que tendremos que dar. Sacamos 10/10, pero no aprendimos nada: solo memorizamos. Con los modelos de machine learning ocurre lo mismo. Si entrenamos y evaluamos con las mismas imágenes, el modelo puede simplemente memorizar los NBI y reportar un error artificialmente bajo. La única forma de saber si realmente aprendió algo general es evaluarlo con imágenes que nunca vio durante el entrenamiento.

Las tres particiones

De acuerdo con el PDF de Tópicos de Implementación, la partición tradicional es 70% entrenamiento, 15% validación y 15% prueba:

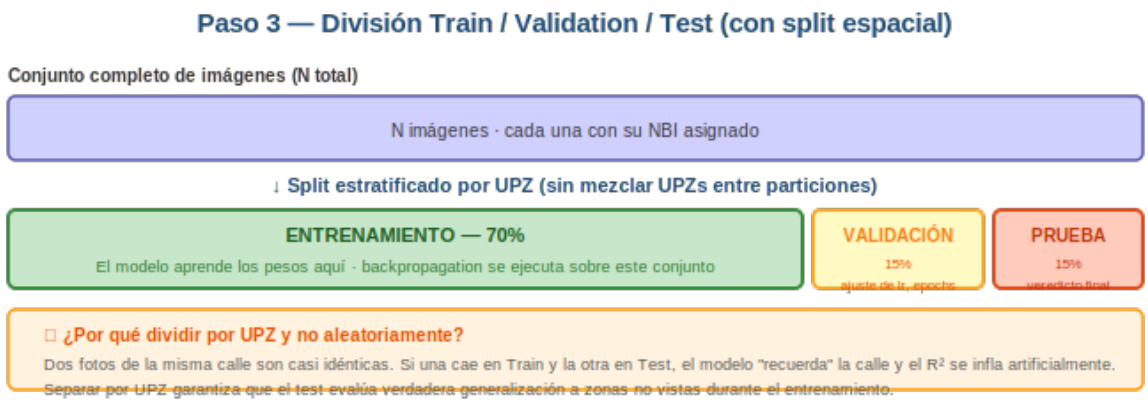


Figura: División train/validation/test con split espacial por UPZ

- Entrenamiento (70%): el modelo aprende aquí. Backpropagation actualiza los pesos solo con estas imágenes.
- Validación (15%): se usa para monitorear el entrenamiento época a época y ajustar hiperparámetros como la tasa de aprendizaje o cuándo parar. El modelo NO aprende de estas imágenes.
- Prueba (15%): se toca una única vez, al final, para reportar el desempeño real del modelo. Es el 'veredicto final'.

¿Qué es el split espacial y por qué es crítico aquí?

Si dividiéramos las imágenes aleatoriamente, podríamos tener dos fotos de la misma calle, tomadas con 51 metros de diferencia, donde una cae en entrenamiento y la otra en prueba. Esas dos imágenes son prácticamente idénticas. El modelo 'recuerda' la calle y el R^2 se infla artificialmente: parece que predice bien, pero en realidad solo reconoce escenas que ya vio.

Para evitar este problema, dividimos por UPZ: todas las imágenes de una UPZ dada van al mismo subconjunto. Esto garantiza que el set de prueba evalúa verdadera generalización a zonas geográficas no vistas durante el entrenamiento, que es exactamente lo que queremos medir.

Analogía econométrica

El split espacial es análogo a usar variación entre municipios para identificación causal, en lugar de variación dentro del mismo municipio donde los observables son demasiado similares. Separar por UPZ evita el equivalente al data leakage que ocurriría si una variable de control y la variable de resultado midieran esencialmente lo mismo.

PASO 4 El Modelo — ResNet18 con Transfer Learning

¿Qué es una red neuronal convolucional (CNN)?

Una red neuronal es un sistema matemático que imita vagamente el cerebro: recibe información (una imagen), la procesa a través de capas sucesivas de transformaciones, y produce un número de salida

(el NBI predicho). Las redes convolucionales están diseñadas específicamente para imágenes: en lugar de procesar todos los píxeles a la vez, aplican pequeños filtros que detectan patrones locales — bordes, texturas, formas, objetos — de manera jerárquica.

En el PDF de Aprendizaje Profundo (Riascos), la propagación hacia adelante (forward propagation) en una red de L capas sigue el esquema general:

```
        z[l] = W[l] · a[l-1] + b[l]     ← combinación lineal
  a[l] = g[l](z[l])                ← función de activación (ReLU)

                donde:  W[l] = pesos de la capa l
                        b[l] = sesgo de la capa l
                       g[l] = ReLU(z) = max(0, z)
```

Este proceso se repite 18 veces en ResNet18, construyendo representaciones cada vez más abstractas: de píxeles → bordes → texturas → formas → objetos → conceptos complejos como 'fachada deteriorada' o 'pavimento en mal estado'.

¿Qué es el Transfer Learning y por qué lo usamos?

Entrenar ResNet18 desde cero requeriría millones de imágenes y semanas de cómputo en GPU. En cambio, usamos una versión ya entrenada por Google en ImageNet: 1.2 millones de fotografías de 1,000 categorías distintas. En ese proceso, la red aprendió representaciones visuales universales que son útiles para cualquier tarea de visión.

El transfer learning aprovecha ese conocimiento. Congelamos las primeras 15 capas (sus pesos no se modifican) y solo actualizamos las últimas 3 capas, incluyendo una capa de salida completamente nueva que reemplaza la original. Esta capa nueva produce un único número real (el NBI predicho) en lugar de probabilidades para 1,000 categorías.

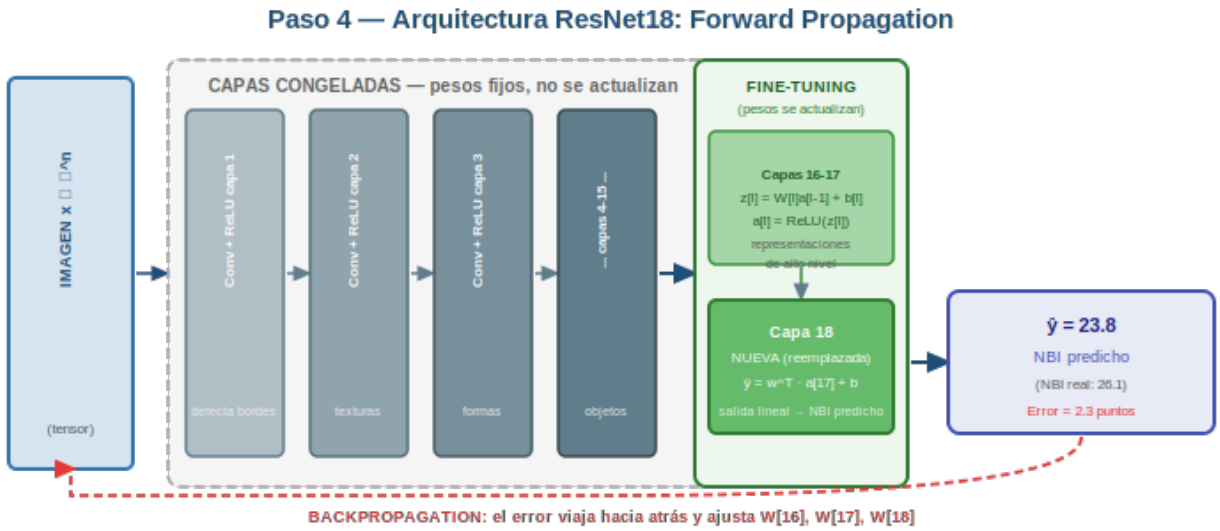


Figura: Arquitectura ResNet18: capas congeladas + fine-tuning + nueva capa de salida

La función de pérdida para regresión — MSE

En clasificación se usa la entropía cruzada como función de pérdida (vista en el PDF 1 de Riascos). En nuestro problema de regresión —predecir un número continuo como el NBI— usamos el Error Cuadrático Medio (MSE):

$$J(W, b) = (1/m) \cdot \Sigma_i (\hat{y}^{(i)} - y^{(i)})^2$$

donde: $\hat{y}^{(i)}$ = NBI predicho para la imagen i
$y^{(i)}$ = NBI real de la zona de la imagen i
m = número de imágenes en el batch

El MSE penaliza más los errores grandes (al elevarlos al cuadrado). Si el modelo predice NBI = 20 y el real es 26, el error es $(20-26)^2 = 36$. Si predice 24, el error es solo 4. El modelo aprende a minimizar esta función.

Backpropagation: cómo aprende el modelo

Una vez calculada la pérdida J, el algoritmo de backpropagation la 'propaga hacia atrás' por la red usando la regla de la cadena del cálculo, calculando cuánto contribuye cada peso W[l] al error total. Ese gradiente indica en qué dirección y cuánto hay que modificar cada peso para reducir el error. Luego, el algoritmo Adam (no lo vimos en clase pero es una versión mejorada del gradiente descendiente) actualiza los pesos:

$$W[l] \leftarrow W[l] - \alpha \cdot \partial J/\partial W[l]$$

donde α es la tasa de aprendizaje (learning rate)

Para la capa L (salida): $dz[L] = a[L] - y$
Para capas internas $l < L$: $dz[l] = W[l+1]^T \cdot dz[l+1] * g'[l](z[l])$

Este ciclo — forward pass → calcular pérdida → backward pass → actualizar pesos — se repite miles de veces, una vuelta por cada batch de imágenes, hasta que el modelo converge.

PASO 5 Entrenamiento, Regularización y Early Stopping

El ciclo de entrenamiento — épocas y batches

Entrenar la red consiste en presentarle las imágenes de entrenamiento repetidamente. Cada vez que el modelo ve todas las imágenes de entrenamiento una vez se llama una época. En la práctica, las imágenes no se presentan de una en una sino en grupos llamados batches (por ejemplo, 32 imágenes a la vez), lo que se llama mini-batch gradient descent. Esto equilibra la estabilidad del gradiente con la velocidad de cómputo.

El problema del sobreajuste (overfitting)

A medida que entrenamos el modelo durante más épocas, su pérdida en el set de entrenamiento siempre baja: está aprendiendo. Pero llega un punto en que empieza a memorizar detalles específicos del set de entrenamiento que no generalizan. Cuando eso pasa, la pérdida de validación empieza a subir aunque la de entrenamiento siga bajando. Ese es el sobreajuste.

Figura: Curvas de pérdida: entrenamiento vs. validación. El área naranja es la zona de sobreajuste.

Regularización 1 — Weight Decay

La primera defensa contra el sobreajuste es el weight decay, discutido en el PDF de Implementación. Agrega un término de penalización a la función de pérdida que castiga los pesos grandes:

```
J_R = J + (ω/2m) · Σ_l ||W[l]||²_F

El gradiente modificado es:
dW[l]_R = dW[l] + (ω/m) · W[l]

La actualización queda:
W[l] ← (1 - α·ω/m) · W[l] - α · dW[l]

El factor (1 - α·ω/m) < 1 'encoge' los pesos en cada paso → weight decay
```

Intuitivamente: penalizar pesos grandes obliga a la red a distribuir la información entre muchas neuronas en lugar de depender excesivamente de unas pocas. Es el análogo exacto de la regularización Ridge (L2) en regresión lineal.

Regularización 2 — Dropout

El dropout (también en el PDF de Implementación) desactiva aleatoriamente una proporción p de neuronas en cada paso del entrenamiento. Según Riascos: 'Dropout distribuye la importancia de las neuronas impidiendo que una tenga demasiado peso.' Durante la evaluación, todas las neuronas se activan pero sus salidas se reescalan:

```
Durante entrenamiento: se apagan p·n neuronas aleatoriamente
Durante evaluación:    z[l] = W[l] · (a[l] / p) + b[l]

El factor 1/p compensa que durante el entrenamiento
la capa tenía menos neuronas activas en promedio
```

Regularización 3 — Early Stopping

Early stopping monitorea la pérdida de validación en cada época y detiene el entrenamiento cuando esa pérdida deja de mejorar por un número de épocas consecutivas (patience). Se guarda el modelo con la mejor pérdida de validación observada, no el del final del entrenamiento. Como nota el PDF de Implementación, esta estrategia mezcla dos objetivos (minimizar la función de costo y evitar el sobreajuste), por lo que la alternativa más robusta es usarla junto con un set de validación separado, exactamente como hacemos aquí.

Resumen del Paso 5

El entrenamiento es un ciclo de ver imágenes → calcular el error → propagar el error hacia atrás → ajustar los pesos. Weight decay penaliza pesos grandes. Dropout apaga neuronas aleatoriamente. Early stopping detiene el entrenamiento en el momento óptimo. Los tres mecanismos actúan juntos para que el modelo aprenda patrones reales y no memorice el ruido de los datos de entrenamiento.

PASO 6 Interpretabilidad — Grad-CAM y Valores SHAP

¿Por qué la interpretabilidad es especialmente importante en economía?

Un modelo que predice bien pero no se puede explicar tiene valor limitado en investigación económica. Si el modelo dice que una zona tiene NBI = 28 pero no podemos saber por qué, no podemos defender ese resultado ante un comité académico, ni identificar políticas basadas en él. La interpretabilidad responde dos preguntas: ¿qué parte de la imagen usó el modelo? (Grad-CAM) y ¿cuánto contribuyó cada característica aprendida a la predicción? (SHAP).

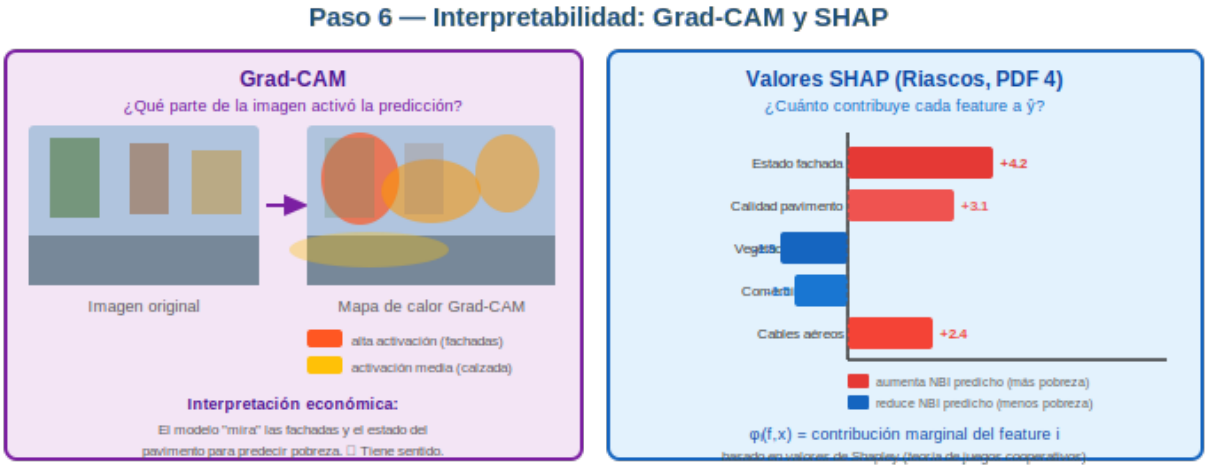


Figura: Grad-CAM: mapa de calor sobre la imagen. SHAP: contribución de cada feature a la predicción.

Grad-CAM — ¿Dónde miró el modelo?

Gradient-weighted Class Activation Mapping (Grad-CAM) calcula el gradiente de la predicción con respecto a los mapas de activación de la última capa convolucional. Esos gradientes indican qué neuronas 'se activaron más' para esa predicción. Al proyectarlos sobre la imagen original, obtenemos un mapa de calor que muestra qué zonas de la fotografía fueron más importantes para el modelo.

```
α_k = (1/Z) · Σ_i Σ_j (∂ŷ / ∂A^k_ij)     ← importancia del mapa k

Mapa Grad-CAM = ReLU( Σ_k α_k · A^k )    ← combinación ponderada
```

```
donde A^k_ij es la activación en la posición (i,j) del mapa k
```

Si el modelo está aprendiendo características económicamente sensatas, el mapa de calor debe activarse sobre fachadas, pavimento, infraestructura visible — no sobre el cielo o marcas de agua de Google. Grad-CAM permite verificar esto visualmente para cualquier imagen.

Valores SHAP — ¿Cuánto contribuyó cada feature?

Los valores SHAP (SHapley Additive Explanations) provienen de la teoría de juegos cooperativos. El PDF de Interpretabilidad de Riascos los presenta como el único concepto de solución que satisface simultáneamente eficiencia, simetría, nulidad y aditividad. La idea es: si la predicción es un 'premio' que deben repartirse las variables del modelo, ¿cuánto le corresponde a cada una?

```
φᵢ(f, x) = Σ_{S ⊆ N\{i}}  [|S|!(|N|-|S|-1)! / |N|!] · [f(S∪{i}) -
                                f(S)]

              donde: φᵢ  = valor SHAP del feature i
                      S   = subconjunto de features sin incluir i
                   f(S)= predicción usando solo los features en S
                       N   = conjunto total de features

    Interpretación: φᵢ es la contribución marginal promedio del
                            feature i
   considerando todos los posibles órdenes de entrada al modelo.
```

En la práctica, para una red neuronal con cientos de features, se usan aproximaciones (DeepSHAP, GradientSHAP). El resultado es un valor por feature por imagen: positivo significa que ese feature elevó la predicción de NBI (más pobreza), negativo que la redujo. Al promediar sobre todo el set de prueba obtenemos la importancia global de cada característica visual aprendida.

Conexión directa con el PDF de Interpretabilidad

Riascos presenta la aplicación de SHAP a CGANs de predicción de crimen en Bogotá. Nuestro ejercicio es metodológicamente idéntico: un modelo de visión computacional aplicado a un fenómeno socioeconómico bogotano, interpretado con los mismos valores de Shapley. La única diferencia es la variable dependiente (crimen vs. NBI) y el tipo de imágenes (sintéticas vs. Street View).

¿Qué esperamos ver en los resultados?

Si el modelo aprendió algo económicamente sensato:

- Grad-CAM activará sobre fachadas, materiales de construcción, estado del pavimento y presencia de infraestructura — no sobre el cielo o el horizonte.
- SHAP mostrará qué features asociados a deterioro urbano (proporciones altas de píxeles en ciertos colores o texturas) contribuyen positivamente al NBI predicho.
- Features asociados a zonas más prósperas (presencia de vegetación urbana, locales formales, señalización) contribuirán negativamente.

Un resultado negativo (modelo que mira zonas irrelevantes) también es informativo: indicaría que las imágenes de vías principales de USME no contienen suficiente señal sobre la condición socioeconómica de los barrios adyacentes.

Síntesis: Los Seis Pasos en Una Página

Paso	Nombre	¿Qué hace?	Concepto del curso
1	**Cruce Geográfico**	Asigna a cada imagen el NBI de su UPZ → construye $\tau=\{(x,y)\}$	*Conjunto de entrenamiento τ (PDF 1)*
2	**Dataset + Augmentation**	Convierte imágenes en tensores normalizados · genera versiones modificadas para ampliar el dataset	*Data augmentation como regularización (PDF 3)*
3	**Split Train/Val/Test**	Divide las imágenes en 70/15/15% por UPZ para evaluación honesta	*División de datos y sesgo vs. varianza (PDF 3)*
4	**ResNet18 Fine-tuning**	Reutiliza una CNN preentrenada · reemplaza la capa final · forward prop: $z[l]=W[l]a[l-1]+b[l]$, $a[l]=ReLU(z[l])$	*Redes profundas, forward propagation (PDF 2)*
5	**Entrenamiento + Regulariz.**	Minimiza MSE con backprop · aplica weight decay ($J_R=J+\omega/2m\cdot\|\|W\|\|^2$), dropout y early stopping	*Weight decay, dropout, early stopping (PDF 3)*
6	**Grad-CAM + SHAP**	Mapa de calor sobre la imagen (¿qué vio?) · valores SHAP $\varphi_i(f,x)$ (¿cuánto contribuyó cada feature?)	*SHAP y valores de Shapley (PDF 4)*